

Deliverable 2

SENG3011 22T1

Testing Documentation

Team f0b5 – Matthew Brian, Jaeff Hong, Mathieu Paturel, Nicholas Ramsay and Huiyi Wang

Contents

Testing Instruments.....	3
Environment.....	3
Tools.....	3
Limitations.....	4
Testing Process.....	4
Methodology	4
Location to GeoID.....	4
Flask Server.....	4
SpaCy.....	5
Scrapy.....	5
Test Cases and Data.....	5
Flask Server.....	5
SpaCy.....	6
Scrapy.....	7
Test Results.....	8

In Testing Documentation report, describe the testing processes used in the development of API, referring to the data and scripts included in Phase_1 folder.

Testing Instruments

Environment

Describe your testing environment

We utilise **GitHub Actions** to implement a form of CI/CD for testing. Before allowing pull requests into the main branch to be merged, that branch must pass the full suite of tests as described pipeline YAML file. This allows tests to be performed remotely and automatically on every branch push to GitHub, preventing broken branches from degrading the quality of the deployed codebase. Each pipeline spawned by GitHub for testing is created using an environment as similar as possible to the production server. Furthermore, if a test does fail, then the *Action* will provide information regarding what failed, making it quick to fix. Tests *can also be performed in a local testing environment*.

```

===== test session starts =====
platform linux -- Python 3.8.10, pytest-7.1.0, pluggy-1.0.0 -- /home/tanya/SENG3011_f0b5/bin/python3
cachedir: .pytest_cache
rootdir: /home/tanya/SENG3011_f0b5/PHASE_1/TestScripts
collected 16 items

test_api.py::test_article_filter_missing_param PASSED
test_api.py::test_report_filter_missing_param PASSED
test_api.py::test_article_invalid_start_date PASSED
test_api.py::test_report_invalid_start_date PASSED
test_api.py::test_article_invalid_date_range PASSED
test_api.py::test_report_invalid_date_range PASSED
test_api.py::test_article_invalid_date_expression PASSED
test_api.py::test_report_invalid_date_expression PASSED
test_api.py::test_article_invalid_timezone_expression PASSED
test_api.py::test_report_invalid_timezone_expression PASSED
test_api.py::test_missing_url PASSED
test_api.py::test_malformed_url PASSED
test_api.py::test_article_too_many_requests PASSED
test_api.py::test_report_too_many_requests PASSED
test_api.py::test_from_article_url_too_many_requests PASSED
test_api.py::test_url_not_found_error PASSED

===== 16 passed (1.12s) =====

```



Tools

Describe testing tools used

As previously described we use the **Pytest** module to validate our server endpoints (via 'integration testing'), scraper, and parser. Pytest is the de facto standard for Python-based user-friendly testing making it widely supported and easy to use. Furthermore, as is the justification for many of the packages selected for this project, we have used it before extensively and are comfortable with its use.

As specified in D1, we considered using the unittest package instead, but it 'requires boilerplate testing classes to be manually written'. Pytest abstracts this through decorators and its much closer integration with testing Flask endpoints. More packages like *DocTest* and *Testify* were considered but they all offered features beyond the scope of what we required for this project.

Postman was used to test the internal server error (500) since this should not occur under normal circumstances. Postman helps us to debug the API implementation and test the error messages manually before going into the integrated Pytest.

We have used custom scripts to stress test our application (concurrent deployments, high number of requests).

Limitations

Describe the limitations of your tests

There are several things we are unable to test. The first is whether the 500 endpoint is correctly returned on an *internal server error*. To ensure that internal server error returns the correct message to the user, we must manually test this error by calling the endpoint with unresolved bugs via Postman in the earlier stages. Unfortunately, even though we know 500 returns the expected error message, we could not test it using Pytest and GitHub Actions.

The second is in testing our parser. It is incredibly difficult to 'test' this kind of parsing program in a life-like environment due to the wide range of possible inputs and their desired outputs. To test if the scraper/parser were working, a pre-existing database with all article info would need to be made and then compared with what the scraper/parser was outputting. This was an unsurmountable problem; such a database would have to be made manually created and then updated daily. This would defeat the purpose of automating scraping and parsing as in testing the scraper we would have made it obsolete. Thus, we compromised by testing the scraper and parser using small, controlled input samples created by us.

We also only test scraping with 10 articles since it takes about 30 min to scrape the entire website like we do in production. So, there may be more issues when deployed.

Testing Process

Methodology

Location to GeoID

To test if the conversion of location names to GeoNames 'geoID' was working correctly, a simple call to the function and its return value could be compared with the known geoID of the location. Simple tests were also performed using fake locations where the function should return -1. Alternative names for countries were also considered, for example, inputting "South Korea" and "Korea" would both produce the same geoID.

```
geo_id = find_GeoID("Hopefully not a v\n")\nassert geo_id == -1\n\ngeo_id = find_GeoID("abcdefghijklmno\n")\nassert geo_id == -1\n\ngeo_id = find_GeoID("New South Wales")\nassert geo_id == 2155400\n\ngeo_id = find_GeoID("China")\nassert geo_id == 1814991
```

Flask Server

To test the server endpoints, we first decided to verify the functioning of all non 200 status codes. This was done by using the in-built 'test_client' function of Flask which allows us to create an instance of server app for testing and helps us to integrate the API tests into the pipeline without interfering with the running server. For 400 status codes this involved feeding in invalid or missing (required) parameters and seeing what was happening. For 404 we called a non-existent endpoint. For 429 this involved spamming the server with API requests. How we test the 500 code with Postman was detailed at the top of this page. The status codes and the types of tests written to call them are in the next section.

Testing Methodology continues on the next page

SpaCy

To test SpaCy, we selected random simple-to-parse articles from CIDRAP. We then parsed them, and then tested to ensure SpaCy gets the same result as what we desired. We are also performing unit tests on the integrated date parsing library.

Scrapy

To ease scraper testing, the scraper itself was modified to accept two more arguments. The first argument was an integer that defined how many pages, starting from the first should be scraped (all pages if -1). This was included prevent individual tests from scraping the entirety of CIDRAP, thus making them much faster. The second argument was the file that the scraper should output to. Without this the scraper would output into only one file which would overwrite production-style runs of the scraper.

Test 1

First, we decided to test whether scraped articles were in the correct format. For our design, the scraper should return a JSON object containing the URL, date of publication, headline, and raw html of an article.

Test 2

Our current scraping implementation stops when it finds a duplicate webpage in the output file. This prevents duplicate articles from appearing in the JSON file. It also allows the system to perform faster after first being run. The next test checks this.

Test Cases and Data

Describe your testing process i.e. how your team conducts testing using the test data and an overview of test cases, testing data and testing results.

Flask Server

What follows is a list of all tests written for the flask server.

Endpoint	Status Code	Status Code Caused By?
article/filter	400	Missing location
		Missing key terms
		Missing start/end date
		Missing two or more
		Missing all params
		Start date later then today
		Start date later then end date
		Date expression not in format "xxxx-xx-xxTxx:xx:xx"

		Time zone expression not in UTC format
		Invalid location
	429	Spamming endpoint with requests
report/filter	400	Missing location
		Missing key terms
		Missing start/end date
		Missing two or more
		Missing all params
		Start date later then today
		Start date later then end date
		Date expression not in format "xxxx-xx-xxTxx:xx:xx"
		Time zone expression not in UTC format
		Invalid location
	429	Spamming endpoint with requests
report /from_article_url	400	Missing URL
		Invalid URL
	429	Spam endpoint with requests
N/A	404	Call a non-existent endpoint on the server
	500	Not forceable

SpaCy

Four tests have been written to test the SpaCy parser. These are described below:

Name	Functionality
Simple Report	Is the parser able to extract report information from a basic article
Ignores Dates After Article Date	Is the parser able to ignore impossibly reported dates
Selects Only Valid Dates	Does the parser only extract relevant dates
Parser Removes Duplicate Locations	Does the parser only extract unique locations

Scrapy

Formatting Test

The formatting test first runs the scraper by a subprocess and waits for the output to be output into a JSON testing file. It then checks for each object in the JSON, and whether the keys 'url', 'date_of_publication', 'headline' and 'article_text' exist. It then checks the format of the URL and date as follows.

```
{
  "url": "/news-perspective/2022/03/stewardship-res",
  "date_of_publication": "2022-3-11 xx:xx:xx",
  "headline": "Resistance Scan for Mar 11, 2022",
  "article_text": "<div>example</div>"
}
```

Each article's URL follows the same format, with '/news-perspective/' at the very beginning, followed by the year and month the article was published. Thus we test URL formatting using a regex to check the beginning characters.

Next, we check the date of publication. Each article's date of publication follows the same format of 'year:month:day xx:xx:xx'. The year is always 4 digits, the month will be 1 digit if the month can be displayed as 1 digit or 2 digits if the month can only be displayed as 2 digits. However, the day is always 2 digits, regardless of whether the day is a 1-digit number. The month can also only be the numbers from 1-12 inclusive whilst the day can only be the numbers from 1-31 inclusive. Again, regex was used, however the regex will accept dates such as 2022:2:31 even though February 31 is not a valid date.

However, we did not think this to be much of a problem as having a date in the correct format, somewhat implied the date would be valid since having a date meant scrapy scraped the date from the website correctly. Testing if the 'headline' and 'article_text' had the *correct format* wasn't nearly as technical. We simply checked if they existed in the JSON Object. This is the end of the first test.

Duplicate Articles Test

The test first runs the scraper via a subprocess and sends its output into a JSON file. Then the contents of this are read into an array, and another array is created just to store the URL of each article. After removing the last article from the JSON file, we run the scraper again to simulate a new article being posted by CIDRAP. If the scraper appended the new article correctly, every URL in the array should be represented by a corresponding JSON object.

```
url_list = [first_article url, second_article url, third_article url, fourth_article url, fifth_article url]

article_list = [second_article, third_article, fourth_article, fifth_article, first_article]
```

Test Results

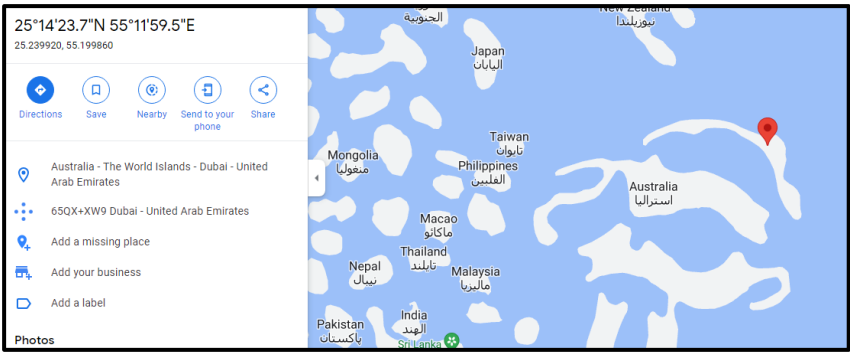
Describe the output of testing and what actions you took to improve the test results.

GeoNames Database

Whilst testing the function that converts locations to geoID, there were many locations that had the same name as other locations with different geoIDs. An example is New South Wales.

15317302	New South Wales	11886915	25.23992	55.19986
15317303	New South Wales	2155400 -33	146	XXXXXXXXXXXXXXXXXXXX

A New South Wales with geoID of 11886915 and another with 2155400.



New South Wales is in Arabia??

To fix this issue, the countries with population of 0 were filtered out of the database. Although this did not completely fix the issue, it removed many irrelevant locations.

CountryToGeoID2-sorted.txt	3/17/2022 7:20 PM	Text Document	511,876 KB
CountryToGeoID-sorted.txt	3/17/2022 3:46 PM	Text Document	8,211,579 KB

This reduced the GeoIDs file size from 8 GB to 0.5 GB, drastically improving testing and production speeds.

Changing Scrapy

Whilst implementing the second Scrapy test, there was a problem which required further change to the behaviour of the scraper. The problem was involved with concurrency where the first request for scraping an article did not necessarily mean it would be first to output into the file. This caused the test to fail in cases where the very first article on the website was not outputted into the JSON file first, and so this article would not be removed from the test's article list since the test removes the very first article in the JSON file.

When the scraper is run again in the test, it would stop at the very first article as the first article is still in the JSON file and the removed article would not be appended in. The change in the scraper was to therefore, not stop at a duplicate article but instead, simply not add that article into the JSON and continue. This change was implemented only for tests by using the fact that *only* tests specify how many pages to scrape. Thus, the scraper would only need to check if the argument for number of pages is not -1.